

Python, pandas, Matplotlib in ARSO

Urejeni zapiski iz pogovora

Zapiski združujejo vprašanja in rešitve iz pogovora: delo s pandas DataFrame-i, risanje grafov, parametrične krivulje s CubicSpline ter pridobivanje vremenskih podatkov z ARSO.

1. Iteriranje po vrsticah DataFrame-a

Metoda `iterrows()` pri vsaki iteraciji vrne dva elementa: indeks vrstice in vrstico kot pandas Series.

```
for indeks, vrstica in df.iterrows():
    print(indeks, vrstica)
```

Če indeksa ne potrebujemo, uporabimo `_.iterrows()`. To ni dodatna spremenljivka, ampak običajna konvencija za vrednost, ki je ne želimo uporabiti.

```
for _, vrstica in df.iterrows():
    print(vrstica)
```

2. Iteriranje po stolpcih

Imena vseh stolpcev dobimo z `df.columns`.

```
for col in df.columns:
    print(col)
```

Če želimo posamezen stolpec in njegove vrednosti:

```
for col in df.columns:
    print(col)
    print(df[col])
```

Če želimo iti skozi vse vrednosti vseh stolpcev:

```
for col in df.columns:
    for value in df[col]:
        print(col, value)
```

3. Legenda pri Matplotlib grafu

Legenda se ustvari z `plt.legend()`. Posameznim grafom določimo ime z argumentom `label`.

```
plt.plot(x, y1, label="Temperatura")
plt.plot(x, y2, label="Višina")

plt.legend()
plt.show()
```

Položaj lahko določimo na primer z `plt.legend(loc="upper right")`.

4. Povprečne vrednosti v vrstici

Za povprečje ene vrstice uporabimo `.mean()`.

```
povprecje = df.iloc[3].mean()
```

Povprečje za vsako vrstico dobimo z `axis=1`:

```
povprecja = df.mean(axis=1)
```

Če želimo povprečje samo izbranih stolpcev:

```
povprecje = df.loc[3, ["A", "B", "C"]].mean()
```

5. Združevanje več DataFrame-ov

Če želimo DataFrame-e združiti po vrsticah, uporabimo `pd.concat()`.

```
df = pd.concat([df1, df2, df3], ignore_index=True)
```

Po stolpcih:

```
df = pd.concat([df1, df2], axis=1)
```

■e jih imamo v seznamu:

```
dfs = [df1, df2, df3, df4]
df = pd.concat(dfs, ignore_index=True)
```

■e želimo združevati na podlagi skupnega stolpca, uporabimo `merge()`.

6. Izbiranje podatkov med dvema vrsticama

Pri strukturi, kjer je na začetku bloka vrstica z imenom gorovja in "Temperatura", na koncu pa vrstica z istim imenom in vejicami, je smiselno najprej najti začetni indeks, nato ime gorovja in zaključni indeks.

```
start_idx = df.index[
    df.iloc[:, 1].astype(str).str.contains(
        "Temperatura", na=False
    )
][0]

ime_gorovja = df.iloc[start_idx, 0]

end_idx = df.index[
    (df.index > start_idx)
    & df.iloc[:, 0].astype(str).str.startswith(
        ime_gorovja, na=False
    )
][0]

rezultat = df.loc[start_idx:end_idx]
```

7. Izbiranje podatkov za isti dan

Pri ARSO DataFrame-u so dnevi zapisani v naslovih stolpcev, npr. `Ne 02 CEST`, `Po 08 CEST`, `To 14 CEST`. Vse stolpce za določen dan lahko izberemo s `startswith()`.

```
nedelja = df.loc[:, df.columns.str.startswith("Ne ")]
ponedeljek = df.loc[:, df.columns.str.startswith("Po ")]
torek = df.loc[:, df.columns.str.startswith("To ")]
```

Za avtomatsko obdelavo vseh dni:

```
dnevi = ["Ne", "Po", "To", "Sr", "■e"]

for dan in dnevi:
    podatki_dan = df.loc[
        :, df.columns.str.startswith(dan + " ")
    ]
```

8. Lihe in sode vrstice

Ker `iloc` uporablja položaje od 0 naprej, moramo paziti na razliko med položajem in ■loveškim štejetjem vrstic.

```
# 1., 3., 5., ... vrstica
lihe = df.iloc[0::2]

# 2., 4., 6., ... vrstica
sode = df.iloc[1::2]
```

■e pa govorimo neposredno o indeksih 0, 1, 2, 3, ...:

```
sode_indeksi = df.iloc[0::2] # 0, 2, 4, ...
lihe_indeksi = df.iloc[1::2] # 1, 3, 5, ...
```

9. Količniki istoležnih celic dveh DataFrame-ov

■e imata DataFrame-a enake vrstice in stolpce, lahko istoležne celice delimo neposredno.

```
kolicniki = df1 / df2

# enakovredno:
```

```
kolicniki = df1.div(df2)
```

■e so v DataFrame-u tudi tekstovne vrednosti, jih lahko pretvorimo v števila; nepretvorljive vrednosti postanejo NaN.

```
df1_num = df1.apply(pd.to_numeric, errors="coerce")
df2_num = df2.apply(pd.to_numeric, errors="coerce")

kolicniki = df1_num / df2_num
```

10. Ve■ grafov na isti figuri

Za ve■ lo■enih grafov v isti figuri uporabimo `plt.subplots()`.

```
fig, axs = plt.subplots(2, 1)

axs[0].plot(x1, y1)
axs[0].set_title("Prvi graf")

axs[1].plot(x2, y2)
axs[1].set_title("Drugi graf")

plt.tight_layout()
plt.show()
```

Za mrežo 2 × 2:

```
fig, axs = plt.subplots(2, 2)

axs[0, 0].plot(x1, y1)
axs[0, 1].plot(x2, y2)
axs[1, 0].plot(x3, y3)
axs[1, 1].plot(x4, y4)

plt.tight_layout()
plt.show()
```

■e pa želiš ve■ ■rt na enem samem grafu, samo ve■krat pokli■eš `plt.plot()` na isti figuri.

11. Gladka parametri■na krivulja s CubicSpline

■e imamo seznam 10–20 vektorjev in želimo prikazati njihovo spreminjanje z gladko parametri■no krivuljo, interpoliramo vsako komponento vektorja glede na skupni parameter t .

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.interpolate import CubicSpline

vektorji = np.array(vektorji)

t = np.arange(len(vektorji))

cs_x = CubicSpline(t, vektorji[:, 0])
cs_y = CubicSpline(t, vektorji[:, 1])

t_gladek = np.linspace(
    0, len(vektorji) - 1, 300
)

x = cs_x(t_gladek)
y = cs_y(t_gladek)

plt.plot(x, y)
plt.scatter(vektorji[:, 0], vektorji[:, 1])
plt.show()
```

Klju■na ideja je parametri■na krivulja $(x(t), y(t))$, ne pa nujno y kot funkcija x . To je pomembno, ■e se krivulja lahko vrne nazaj po osi x .

Za 3D vektorje naredimo tri spline funkcije:

```
cs_x = CubicSpline(t, vektorji[:, 0])
cs_y = CubicSpline(t, vektorji[:, 1])
cs_z = CubicSpline(t, vektorji[:, 2])

# dobimo parametri■no krivuljo:
# (x(t), y(t), z(t))
```

12. Pridobivanje podatkov z ARSO spletne strani

Pri sodobnih spletnih straneh klik na drugo gorovje ne pomeni nujno, da se odpre druga HTML stran. Pogoste so tri možnosti: JavaScript pošlje zahtevek na API; vsi podatki so že v HTML/JavaScript kodi; ali JavaScript iz JSON podatkov dinamično zgradi tabelo.

Preverjanje v Chromu: F12 → Network → Fetch/XHR → klik na drugo gorovje → preveri, ali se pojavi nov zahtevek. Če zahtevek vrača JSON, je pogosto najboljša rešitev neposreden klic tega vira iz Pythona.

Če so podatki v HTML:

```
import requests
from bs4 import BeautifulSoup

url = "https://meteo.arso.gov.si/met/sl/weather/bulletin/mountain/"
html = requests.get(url).text

soup = BeautifulSoup(html, "html.parser")

tables = soup.find_all("table")
print(len(tables))
```

Če podatke nalaga JavaScript: ena možnost je Selenium, ki odpre pravi brskalnik.

```
from selenium import webdriver
from selenium.webdriver.common.by import By
import time

driver = webdriver.Chrome()

driver.get(
    "https://meteo.arso.gov.si/met/sl/weather/bulletin/mountain/"
)

time.sleep(3)

driver.find_element(
    By.XPATH,
    "//button[contains(text(),'Julijske Alpe')]"
).click()

time.sleep(2)

print(driver.page_source)
driver.quit()
```

Priporočena možnost: najti skriti API, ki ga uporablja JavaScript, in nato uporabiti requests.

```
import requests

r = requests.get(api_url)
print(r.json())
```

13. Backoff pri spletnih zahtevah

Backoff pomeni, da po neuspešni zahtevi program nekaj časa počaka in nato poskusi znova. To je uporabno, kadar je strežnik začasno nedosegljiv ali preobremenjen.

```
import requests
import time

def prenos_html_strani(seznam_gorovij):
    for gorovje in seznam_gorovij:
        for poskus in range(3):
            try:
                r = requests.get(
                    povezava_gorovje(gorovje),
                    timeout=10
                )

                if r.status_code == 200:
                    vsebina = r.text
                    print(
                        f"{gorovje}: uspešno preneseno"
                    )
                    break
```

```

    else:
        print(
            f"Napaka {r.status_code}"
        )

except requests.RequestException:
    print("Napaka pri povezavi.")

time.sleep(2)

```

V tem primeru so največ trije poskusi. `break` prekine samo notranjo zanko in program nato nadaljuje z naslednjim gorovjem.

Eksponentni backoff pomeni, da se čas čakanja povečuje, na primer 1, 2 in 4 sekunde.

```

import time

for poskus in range(3):
    try:
        r = requests.get(...)
        if r.status_code == 200:
            break
    except requests.RequestException:
        pass

    time.sleep(2 ** poskus)

```

Eksponentni backoff je pogosta bolj robustna strategija pri ponavljanju zahtev do spletnih strežnikov.

Hiter povzetek: za pandas je najpomembneje poznati `iloc/loc`, `iterrows()`, `concat()`, `mean()`, `startswith()` in neposredno razlikovanje med `DataFrame`-i. Za Matplotlib so ključni `plot()`, `legend()`, `subplots()` in pri gladkih parametričnih poteh `CubicSpline`. Pri ARSO podatkih je najprej smiselno preveriti `Network/XHR` in poiskati vir `JSON/API`, preden uporabimo `Selenium`.